CAS CS 538. Solutions to Problem Set 8

Due electronically via Gradescope, Monday April 6, 2026 11:59pm

Om Khadka, U51801771

How to submit: See instructions on Problem Set 1.

Trapdoor Hash Functions. For this problem, we will augment collision-resistant hashing with a useful property.

We will consider keyed collision-resistant hash functions that take a hashing key pk and two input values (which we will call (a, b)), and output one value. We will now augment these hash functions with an additional property: a trapdoor secret key sk that enables easy collision finding.

We formalize this property as follows. We add a key generation algorithm G that outputs a public hashing key pk and a trapdoor key sk. There is also an efficient algorithm T to find collisions given sk: specifically, $T(sk, a_1, b_1, a_2)$, outputs b_2 such that $H(pk, a_1, b_1) = H(pk, a_2, b_2)$. We still require that the hash function remains collision resistant for anyone who is given pk but not sk.

In the next two problems you will build trapdoor hash functions. Of course, if a hash function has a trapdoor, then the person who generated the key will be able to find collisions.

Problem 1. (30 points) Recall the DL-based family in Section 8.5.1 of the textbook. Let q be a Sophie Germain prime, so that $p = 2q + 1$ is also prime. Let $x, y \in QR_p$ and $a, b \in \{1, 2, \ldots, q\}$. The public key is (x, y) and $H((x, y), a, b) = \text{abs}(x^a y^b \bmod p)$, where $\text{abs}(z)$ returns z if $z < p/2$ and $-z$ otherwise.

Show that H is actually a trapdoor hash family. In other words, demonstrate an algorithm G creates $pk = (x, y)$ and some sk (your job is to figure out what sk should be), and algorithm T.

Solution. Given the abstract definition of the trapdoor hash func before Q1, this question is now basically making me show an actual example of how a such a func would work. The main way I will solve this is (basically) through modular arithmetic.

$\mathbb{G}$ and $\mathcal{T}$

Let $\mathbb{G}$ be the key gen for the trapdoor func, $\mathcal{T}$.

$\mathbb{G}()$:

- Select rnd $x \in QR_p \backslash \{1\}$.
- Select rnd $s \in \mathbb{Z}_p^*$.
- Compute y as: $y = x^s \mod p$.
- $pk = (x, y)$, $sk = s$.

$\mathcal{T}(sk, a_1, b_1, a_2)$:

- Parse sk as s.
- Computes b_2 as: $b_2 = b_1 + s^{-1}(a_1 - a_2)$.
- Return b_2.

Correctness

Recall that $H((x,y),a,b) = abs(x^a y^b \mod p)$. There is an important property of this abs func that I need to say first-off:

- Because $x, y \in QR_p$, and QR_p is multiplpicatively closed, then $\forall a, b, x^a y^b \in QR_p$.
- q is odd. That means $p \equiv_4 3$. Note that -1 can only be a quad. residue $\mod p \iff p \equiv_4 1$. Therefore, $-1 \notin QR_p$.
- Therfore, if $z_1, z_2 \in QR_p, abs(z_1) = abs(z_2) \iff z_1 \equiv_p z_2$. Basically, the case that $z_1 \equiv -z_2$ is impossible.
  - Therefore we can just reduce the equation we need to solve from $abs(\cdot) \equiv_p abs(\cdot)$ to just $x^{a_1} y^{b_1} \equiv_p x^{a_2} y^{b_2}$.

Firstly we can just sub y for $x^s \mod p$:

$$x^{a_1+sb_1} \equiv_p x^{a_2+sb_2}$$

Now, from $\mathcal{T}$, we get $b_2 \equiv_q b_1 + s^{-1}(a_1 - a_2)$. By just multiplying both sides w/ s:

$$sb_2 \equiv_q sb_1 + a_1 - a_2 \to a_1 + sb_1 \equiv_q a_2 + sb_2$$

Now, note that x, q are both prime, and that x has order q. That means that exponents of x are equivalent $\mod q$ iff the powers are equivalent modulo p:

$$x^{a_1+sb_1} \equiv_p x^{a_2+sb_2} \to a_1 + sb_1 \equiv_q a_2 + sb_2$$

This equation holds perfectly. It is also the exact same output as from $\mathcal{T}$, which is assumed to output the collision of b_2. Therfore, this shows how $H(pk, a_1, b_1) = H(pk, a_2, b_2)$ if you run $\mathcal{T}(sk, a_1, b_1, a_2)$.

Game

Now for the proof that $\mathbb{G}, \mathcal{T}$ holds, I will make an adversary game, with a MITM. The MITM, $\mathcal{B}$, will try to simluate a discrete log (DL) game for $\mathcal{A}$, which is playing against the CRHF.

Let $\mathcal{A}$ be an eff. adversary, that can find collisions within H, w/ advantage ϵ, when only given pk. $\mathcal{B}$ will then also be another efficent adv., solving the DL problem in QR_p with the same probability.

DL Challenge $\mathcal{B}$ firstly gets $(p, g, \mathcal{Y})$, where g generates QR_p and $\mathcal{Y} \in QR_p$. The goal is to find s such that $g^s \equiv_p \mathcal{Y}$.

Execution $\mathcal{B}$ will set $pk \to (g, \mathcal{Y})$. This distribution is the same as a valid public key, outputted by $\mathbb{G}$. Then, $\mathcal{B}$ will give pk to $\mathcal{A}$. Finally, $\mathcal{A}$ will output $(a_1, b_1), (a_2, b_2)$, such that they cause a collision when both plugged into H.

Extracting the Key By the properties of *abs* from before, we are just solving for:

$$g^{a_1}\mathcal{Y}^{b_1} \equiv_p g^{a_2}\mathcal{Y}^{b_2}$$

Rearrage the terms:

$$g^{a_1-a_2} \equiv_p \mathcal{Y}^{b_2-b_1}$$

Let $\triangle a = (a_1 - a_2)$, and $\triangle b = (b_2 - b_1)$. Then, we're just solving for:

$$g^{\triangle a} \equiv_p \mathcal{Y}^{\triangle b}$$

Since $(a_1, b_1) \neq (a_2, b_2)$, there no way that $\triangle a \equiv_q \triangle b \equiv_q 0$. Imagine if $\triangle b \equiv 0$. Then $g^{\triangle a} \equiv_p 1$. And since g has order q, this forces $\triangle a \equiv_p 0$ as well. This is impossible. So let's just say that $\triangle b \not\equiv_q 0$.

Now, since q is prime, then $\triangle b$ is invertible $\mod q$.Then, $\mathcal{B}$ computes $(\triangle b)^{-1} \mod q$ and gets:

$$s \to \triangle a \cdot (\triangle b)^{-1} \mod q$$

This satisfies $\mathcal{Y} \equiv_p g^s$, therefore this is a DL challenge. $\mathcal{B}$ only wins when $\mathcal{A}$ finds a collision. Since DL is hard, $\mathcal{A}$'s success is neglgible.

This proves the security of the construction of $\mathbb{G}, \mathcal{T}$. $\square$

Problem 2. (40 points at 10 each) In this problem, we will build a collision-resistant hash function based on the hardness of factoring rather on the discrete logarithm problem.

Let q_1 and q_2 be two distinct Sophie Germain primes, so that $p_1 = 2q_1 + 1$ and $p_2 = 2q_2 + 1$ are primes. Let $n = p_1p_2$. Denote by QR_{p_1}, QR_{p_2}, and QR_n the groups of squares modulo p_1, p_2, and n, respectively. We know that QR_{p_1} and QR_{p_2} are cyclic; thus, let $g_1 \in QR_{p_1}$ generate QR_{p_1} and $g_2 \in QR_{p_1}$ generate QR_{p_2}.

(a) Show by CRT (see notes from Discussion 8) that QR_n is cyclic (the easiest way to do this is to try to construct a generator of QR_n and show that it is indeed a generator by showing that its order is q_1q_2).

Solution. Your solution goes here

(b) Show how to efficiently factor n given q_1q_2. In fact, you can factor n given any multiple of q_1q_2, but we won't show it here. Hint: treat q_1 and q_2 as unknowns. Construct a system of equations for these unknowns and solve it.

Solution. Your solution goes here

(c) Let k be the bit length of n. Let g be a generator of QR_n. The public key for our hash function is n and g. The inputs will be two values in the set $\{1, 2, \ldots, q_1q_2\}$. Let $H(a, b) = g^{a2^k+b} \mod n$ (note that $a2^k + b$ is just concatenation of a and b as bit strings). Show that this is a collision-resistant hash family under the assumption that such n are hard to factor. Specifically, show how, given (a_1, b_1) and

(a_2, b_2) that collide, one can factor n. You may use without proof the fact that n can be efficiently factored given any non-zero multiple of $q_1 q_2$.

Solution. Your solution goes here

(d) Show also that this is a trapdoor hash family (like in Problem 1, you need to find what trapdoor information G should output and what the algorithm T will do).

Solution. Your solution goes here

Problem 3. (30 points) Show how to compute the root of an n-leaf Merkle tree in $O(\log n)$ space. More precisely, assume n is a power of 2. Given data items $x_1, x_2, \ldots, x_n$ as values to be placed in the leaves of the tree, and a hash function H, write pseudocode to compute the root r of the tree while never storing more than $2 + \log n$ hash values. Analyze the storage space used by your pseudocode to prove that is never more than $2 + \log n$. Note that we do not count the storage of the data items.

Solution. Your solution goes here